EEP 596 Computer Vision - Assignment 6 Report

Date: November 8, 2025

Task 1: Number of Parameters

Objective: Write a function to compute the number of trainable parameters in a model (e.g., ResNet-34).

```
def compute_num_parameters(net:nn.Module): num_para = sum(p.numel() for p in net.parameters() if p.requires_grad) return num_para
```

Test with ResNet-34:

```
Number of trainable parameters in ResNet-34: 21,797,672
```

Task 2: Global Average Pooling (GAPNet)

Objective: Create and train GAPNet for 10 epochs on CIFAR-10 dataset.

Network Architecture:

```
GAPNet( (conv1): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1)) (pool): MaxPool2d(kernel_size=2, stride=2, padding=0)
```

```
(conv2): Conv2d(6, 10, kernel_size=(5, 5), stride=(1, 1))
(gap): AvgPool2d(kernel_size=10, stride=10, padding=0) (fc):
Linear(in_features=10, out_features=10, bias=True) )
```

Training Parameters:

- Epochs: 10
- Learning Rate: 0.001
- Momentum: 0.9
- Optimizer: SGD

Training Output:

```
Runs on cuda device.
Runs on cuda device.
Runs on cuda device.
[1,  2000] loss: 2.140
[1,  4000] loss: 1.933
[1,  6000] loss: 1.838
[1,  8000] loss: 1.785
[1, 10000] loss: 1.733
[1, 12000] loss: 1.688
Runs on cuda device.
Runs on cuda device.
[2,  2000] loss: 1.653
[2,  4000] loss: 1.623
[2,  6000] loss: 1.623
[2,  8000] loss: 1.600
[2, 10000] loss: 1.573
[2, 12000] loss: 1.566
Runs on cuda device.
Runs on cuda device.
[3,  2000] loss: 1.537
[3,  4000] loss: 1.532
[3,  6000] loss: 1.508
[3,  8000] loss: 1.481
[3, 10000] loss: 1.490
[3, 12000] loss: 1.492
Runs on cuda device.
Runs on cuda device.
[4,  2000] loss: 1.462
[4,  4000] loss: 1.461
[4,  6000] loss: 1.446
[4,  8000] loss: 1.439
```

```
[4, 10000] loss: 1.433
[4, 12000] loss: 1.426
Runs on cuda device.
Runs on cuda device.
[5,  2000] loss: 1.424
[5,  4000] loss: 1.405
[5,  6000] loss: 1.417
[5,  8000] loss: 1.388
[5, 10000] loss: 1.370
[5, 12000] loss: 1.390
Runs on cuda device.
Runs on cuda device.
[6,  2000] loss: 1.374
[6,  4000] loss: 1.360
[6,  6000] loss: 1.372
[6,  8000] loss: 1.367
[6, 10000] loss: 1.361
[6, 12000] loss: 1.370
Runs on cuda device.
Runs on cuda device.
[7,  2000] loss: 1.347
[7,  4000] loss: 1.361
[7,  6000] loss: 1.349
[7,  8000] loss: 1.344
[7, 10000] loss: 1.342
[7, 12000] loss: 1.322
Runs on cuda device.
Runs on cuda device.
[8,  2000] loss: 1.309
[8,  4000] loss: 1.304
[8,  6000] loss: 1.324
[8,  8000] loss: 1.341
[8, 10000] loss: 1.329
[8, 12000] loss: 1.329
Runs on cuda device.
Runs on cuda device.
[9,  2000] loss: 1.312
[9,  4000] loss: 1.295
[9,  6000] loss: 1.324
[9,  8000] loss: 1.305
[9, 10000] loss: 1.314
[9, 12000] loss: 1.294
Runs on cuda device.
Runs on cuda device.
[10,  2000] loss: 1.295
[10,  4000] loss: 1.279
[10,  6000] loss: 1.293
```

```
[10,  8000] loss: 1.291
[10, 10000] loss: 1.306
[10, 12000] loss: 1.298
Finished Training
Runs on cuda device.
Runs on cuda device.
Accuracy on 10000 test images: 53.40%
```

Model Information:

```
Number of trainable parameters in GAPNet: 2,076

Model saved as: Gap_net_10epoch.pth
```

Task 3: Backbones (Feature Extraction)

Objective: Download ResNet-18 (pretrained on ImageNet), remove the final fully connected layer, and extract features for "cat_eye.jpg".

```
def backbone(): model =
models.resnet18(weights=models.ResNet18_Weights.DEFAULT) model
= nn.Sequential(*list(model.children())[:-1]) # Remove final
FC layer model.eval() transform = transforms.Compose([
transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5),
(0.5, 0.5, 0.5)) ]) image =
Image.open('cat_eye.jpg').convert('RGB') image_tensor =
transform(image).unsqueeze(0) with torch.no_grad(): features =
model(image_tensor) return features
```

Feature Extraction Output:

```
Extracted features shape: torch.Size([1, 512, 1, 1])

Feature vector dimensions: [batch_size=1, features=512, height=1, width=1]

Total feature elements: 512
```

Task 4: Transfer Learning with ResNet-18

Objective: Use pretrained ResNet-18, modify the last layer for 10 classes (CIFAR-10), freeze all weights except the last layer, and train for 10 epochs.

Network Modifications:

```
class TransferFromResNet18Model(nn.Module): def __init__(self, num_classes=10): super(TransferFromResNet18Model, self).__init__() resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT) # Freeze all layers for param in resnet.parameters(): param.requires_grad = False # Replace final FC layer for CIFAR-10 (10 classes) resnet.fc = nn.Linear(512, num_classes) self.model = resnet
```

Training Parameters:

- Epochs: 10
- Learning Rate: 0.001
- Momentum: 0.9
- Optimizer: SGD (only last layer)
- Batch Size: 32

Training Output:

```
Runs on cuda device.
Runs on cuda device.
Runs on cuda device.
[1,   200] loss: 2.039
[1,   400] loss: 1.807
[1,   600] loss: 1.724
[1,   800] loss: 1.674
[1,  1000] loss: 1.679
[1,  1200] loss: 1.669
[1,  1400] loss: 1.649
Runs on cuda device.
Runs on cuda device.
[2,   200] loss: 1.631
[2,   400] loss: 1.621
[2,   600] loss: 1.629
[2,   800] loss: 1.644
[2,  1000] loss: 1.630
[2,  1200] loss: 1.610
[2,  1400] loss: 1.601
Runs on cuda device.
Runs on cuda device.
[3,   200] loss: 1.610
[3,   400] loss: 1.617
[3,   600] loss: 1.605
[3,   800] loss: 1.600
[3,  1000] loss: 1.615
[3,  1200] loss: 1.621
[3,  1400] loss: 1.603
Runs on cuda device.
Runs on cuda device.
[4,   200] loss: 1.601
[4,   400] loss: 1.601
[4,   600] loss: 1.593
[4,   800] loss: 1.585
[4,  1000] loss: 1.625
[4,  1200] loss: 1.598
[4,  1400] loss: 1.612
Runs on cuda device.
Runs on cuda device.
[5,   200] loss: 1.591
[5,   400] loss: 1.601
[5,   600] loss: 1.579
[5,   800] loss: 1.605
[5,  1000] loss: 1.581
[5,  1200] loss: 1.617
```

```
[5,  1400] loss: 1.623
Runs on cuda device.
Runs on cuda device.
[6,   200] loss: 1.603
[6,   400] loss: 1.592
[6,   600] loss: 1.620
[6,   800] loss: 1.603
[6,  1000] loss: 1.605
[6,  1200] loss: 1.611
[6,  1400] loss: 1.604
Runs on cuda device.
Runs on cuda device.
[7,   200] loss: 1.579
[7,   400] loss: 1.614
[7,   600] loss: 1.607
[7,   800] loss: 1.611
[7,  1000] loss: 1.615
[7,  1200] loss: 1.608
[7,  1400] loss: 1.610
Runs on cuda device.
Runs on cuda device.
[8,   200] loss: 1.590
[8,   400] loss: 1.596
[8,   600] loss: 1.601
[8,   800] loss: 1.590
[8,  1000] loss: 1.592
[8,  1200] loss: 1.627
[8,  1400] loss: 1.586
Runs on cuda device.
Runs on cuda device.
[9,   200] loss: 1.599
[9,   400] loss: 1.615
[9,   600] loss: 1.577
[9,   800] loss: 1.588
[9,  1000] loss: 1.599
[9,  1200] loss: 1.602
[9,  1400] loss: 1.604
Runs on cuda device.
Runs on cuda device.
[10,   200] loss: 1.597
[10,   400] loss: 1.608
[10,   600] loss: 1.624
[10,   800] loss: 1.605
[10,  1000] loss: 1.594
[10,  1200] loss: 1.612
[10,  1400] loss: 1.606
Finished Training
```

```
Model saved as Res_net_10epoch_gpu.pth
Runs on cuda device.
Runs on cuda device.
Accuracy on 10000 test images: 44.36%
Model weights loaded to CPU.
CPU-version model weights saved to: ./Res_net_10epoch.pth
```

Model Information:

```
Total parameters in model: 11,181,642

Trainable parameters (final layer only): 5,130

Frozen parameters: 11,176,512

Model saved as: Res_net_10epoch.pth
```

Task 5: MobileNet Implementation

Objective: Implement MobileNetV1 in PyTorch with depthwise separable convolutions, batch normalization, and ReLU activation.

Key Components:

- Standard Convolution + BatchNorm + ReLU
- Depthwise Separable Convolution (Depthwise + Pointwise)
- Global Average Pooling
- Fully Connected Classifier

```
class MobileNetV1(nn.Module): def __init__(self, ch_in, n_classes): super(MobileNetV1, self).__init__() def conv_bn(in_channels, out_channels, stride): return nn.Sequential( nn.Conv2d(in_channels, out_channels,
```

```
kernel_size=3, stride=stride, padding=1, bias=False), nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True) ) def conv_dw(in_channels, out_channels, stride): return nn.Sequential( # Depthwise Convolution nn.Conv2d(in_channels, in_channels, kernel_size=3, stride=stride, padding=1, groups=in_channels, bias=False), nn.BatchNorm2d(in_channels), nn.ReLU(inplace=True), # Pointwise Convolution nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, padding=0, bias=False), nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True) ) self.features = nn.Sequential( conv_bn(ch_in, 32, 2), conv_dw(32, 64, 1), conv_dw(64, 128, 2), conv_dw(128, 128, 1), conv_dw(128, 256, 2), conv_dw(256, 256, 1), conv_dw(256, 512, 2), conv_dw(512, 512, 1), conv_dw(512, 512, 1), conv_dw(512, 512, 1), conv_dw(512, 512, 1), conv_dw(512, 512, 1), conv_dw(512, 1024, 2), conv_dw(1024, 1024, 1), nn.AdaptiveAvgPool2d(1) ) self.classifier = nn.Linear(1024, n_classes)
```

Dimension Check:

```
Input shape: [1, 3, 224, 224]

Output shape: [1, 1000]

✓ Dimension check PASSED!

Number of trainable parameters in MobileNetV1: 4,231,976
```

Summary

Task	Model/Component	Status	Output File
Task 1	Parameter Counter (ResNet-34)	✓ Completed	-
Task 2	GAPNet Training	✓ Completed	Gap_net_10epoch.pth
Task 3	ResNet-18 Feature Extraction	✓ Completed	-
Task 4	Transfer Learning (ResNet-18)	✓ Completed	Res_net_10epoch.pth
Task 5	MobileNetV1 Implementation	✓ Completed	-